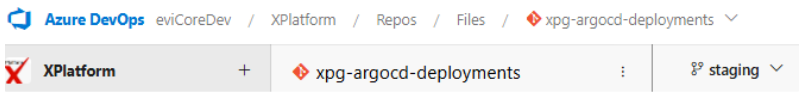
Migration to New AKS ARGO CD Cluster with XPG Apps

As per the ARGO CD deployment process, we should create a common repo for centralizing deployment triggers.

The **xpg-argocd-deployments** repository is designated to contain all deployments of AKS-hosted applications for XPG across all environments associated with the Xplatform Project.



COE official documentation for reference: [Shared AKS Getting Started Guide - Cloud - Confluence]

Recording of walkthrough of AKS migration : [AKS ARGO CD Migration Session]:

Migration Steps:

1. **Eliminate Secrets from CICD Pipeline** Ensure that all secrets are removed from your CICD pipeline and securely transferred to their respective project vaults
2. **Create Managed Identity Resource with Federated Credential** To create a managed identity resource and assign permissions to the vaults using Terraform, follow the example below:

```
resource "azurerm_user_assigned_identity" "example" {
  name                = "example-identity"
  resource_group_name = "example-resources"
  location            = "East US"
}

resource "azurerm_key_vault_access_policy" "example" {
  key_vault_id = azurerm_key_vault.example.id
  tenant_id    = data.azurerm_client_config.example.tenant_id
  object_id    = azurerm_user_assigned_identity.example.principal_id
  secret_permissions = [
    "get",
    "list",
  ]
}
resource "azurerm_federated_identity_credential" "example" {
  name                = "example-federated-credential"
  resource_group_name = "example-resources"
  parent_id           = azurerm_user_assigned_identity.example.id
  audience            = ["api://AzureADTokenExchange"]
  issuer              = "https://eastus2.oic.prod-aks.azure.com/595de295-db43-4c19-8b50-183dfd4a3d06/423f3c78-4672-42b9-8937-ebed1dc38042/" /* this is dev */
  subject             = "system:serviceaccount:{your-sa-name}"
}

```

   Replace `clientID.1156c6a4-5198-4b0c-b987-2e4353fb1d21` with your managed identity client ID and `eheu2dv1akvocr` with your key vault name

3. **Adding a Secret Provider YAML**

   To add a Secret Provider YAML, follow these steps:

   Include the Secret Name: In the object array provided in the code snippet, include the secret name that you created in Vault as mentioned in step #1.

   Case Sensitivity: Please note that `objectName` is case-sensitive.

   Matching Object Names: Ensure that the `objectName` in the “objects” section matches exactly with the `objectName` in the “secretObjects” section.

   Key Vault Name: The `keyvaultName` should correspond to the name of your vault.

By adhering to these guidelines, you can successfully configure your Secret Provider YAML.

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
```

```yaml
  name: xpg-workload-identity-sa
  namespace: xpg-poc
  annotations:
    azure.workload.identity/client-id: 1156c6a4-5198-4b0c-b987-2e4353fb1d21
---
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: xpg-azure-kv-secret-provider
  namespace: xpg-poc
spec:
  provider: azure
  parameters:
    usePodIdentity: "false"
    clientID: "1156c6a4-5198-4b0c-b987-2e4353fb1d21"
    keyvaultName: "eheu2dv1akvocr"
    objects: |
      array:
        - |
          objectName: OCRClientSecret
          objectType: secret
        - |
          objectName: SecuredSaslPassword
          objectType: secret
        - |
          objectName: kafka-certs-dev
          objectType: secret
    tenantId: "595de295-db43-4c19-8b50-183dfd4a3d06"
  secretObjects:
  - secretName: ocrclientsecret
    type: Opaque
    data:
    - key: ocrclientsecret
      objectName: OCRClientSecret

  - secretName: securedsaslpassword
    type: Opaque
    data:
    - key: securedsaslpassword
      objectName: SecuredSaslPassword

  - secretName: kafka-certs-dev
    type: Opaque
    data:
    - key: kafka-certs-dev
      objectName: kafka-certs-dev
```

Eg. [secretprovider.yml - Repos]

4. **Addition in Deployment yaml**

Add the yellow-marked labels and spec in your deployment. yaml for accessing the vault via MI

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ocr-receptorapi
spec:
  replicas: {#ReceptorMinReplicasSpecs#}
  revisionHistoryLimit: ...
  selector:
    matchLabels:
      app: ocr-receptorapi # Deployment label for pod and s
  template:
    metadata:
      labels:
        azure.workload.identity/use: "true"
        app: ocr-receptorapi # Pod label. This must match to
        service: ocr
    spec:
      serviceAccountName: xpg-workload-identity-sa
      containers:
        - image: {#ContainerImageOCRReceptor#}
          imagePullPolicy: Always
          name: ocr-receptorapi
```

Add your secret as an env variable in deployment.yaml for accessing secrets in c# in below manner

```yaml
      containers:
        - image: {#ContainerImageOCRReceptor#}
          imagePullPolicy: Always
          name: ocr-receptorapi
          ports:
            - ...
            - containerPort: 80 # Container port open to access. This must b
              name: http
          resources:
            ...
            limits:
              memory: "{#OCRReceptorMemory#}"
              cpu: "{#OCRReceptorCpu#}"
          env:
            - name: ObjectValet__ClientSecret
              valueFrom:
                secretKeyRef:
                  name: ocrclientsecret
                  key: ocrclientsecret
            - name: Authentication__OcrOAuthSecret
              valueFrom:
                secretKeyRef:
                  name: ocrclientsecret
                  key: ocrclientsecret
            - name: Kafka__SaslPassword
              valueFrom:
                secretKeyRef:
                  name: securedsaslpassword
                  key: securedsaslpassword
            - name: AZURE_CLIENT_ID
              value: "1156c6a4-5198-4b0c-b987-2e4353fb1d21"
          envFrom:
            - configMapRef:
                name: ocr-receptorapi-configmap
          volumeMounts:
            - name: certificate-vol
              mountPath: /certs
              readOnly: true
      imagePullSecrets:
        - name: ocr-receptorapi-docker-secret
      volumes:
        - name: certificate-vol
          csi:
            driver: secrets-store.csi.k8s.io
            readOnly: true
            volumeAttributes:
              secretProviderClass: "xpg-azure-kv-secret-provider"
```

5. **Changes in Ingress.yaml if your app is an API**.

Add the yellow-marked values in your ingress.yaml for accurate attribute routing accuracy

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ocr-receptor-ingress
  namespace: {#Namespace#}
  annotations:
    kubernetes.io/ingress.class: "nginx"
    nginx.org/mergeable-ingress-type: "minion"
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/x-forwarded-prefix: /ocr-receptorapi
spec:
  rules:
  - host: {#AKSCluster#}
    http:
      paths:
      - path: "/ocr-receptorapi(/|$)(.*)"
        pathType: Prefix
        backend:
          service:
            name: ocr-receptorapi
            port:
              number: 80    # Requests coming to above path is redirected to this port against the
```

6. AKS Onboarding Process

Raise an AKS Onboarding SNOW

- **Request**: Raise an AKS Onboarding SNOW as requested for your application with new namespace.
- **Sample POC SNOW**: SNOW for AKS Cluster Migration [SNOW for AKS Cluster Migration]

7. Deployment of the Manifest

Description

This PowerShell script is specifically crafted to commit the final YAML file to the Argo CD repository. It is intended to be executed following the completion of the tokenization task.

PowerShell Script for Committing to Argo CD Repository

Action

The attached script in the right will facilitate the commit of the final YAML file to the Argo CD repository in the **staging** branch by default.

Please make sure you have added ACR Pull Role on your ACR on AKS Managed or User Assigned Identity before you deploy to ARGOCD cluster

- PowerShell Command let for **ACRPull** Role

```powershell
# Example data
$resourceGroupName = "xpg_shared_infra"
$containerRegistryName = "eheu2xpgsharedacr"
$identityName = "eh-aks-1-dev-agentpool"
$roleDefinitionName = "AcrPull"

# Get the Container Registry ID
$scope = (Get-AzContainerRegistry -ResourceGroupName $resourceGroupName -Name $containerRegistryName).Id

# Get the User Assigned Identity ID
$principalId = (Get-AzUserAssignedIdentity -ResourceGroupName $resourceGroupName -Name $identityName).PrincipalId

# Create the Role Assignment
New-AzRoleAssignment -Scope $scope -RoleDefinitionName $roleDefinitionName -PrincipalId $principalId

```

- Terraform for **ACRPull** Role

```terraform
resource "azurerm_role_assignment" "example" {
  scope                = azurerm_container_registry.example.id
  role_definition_name = "AcrPull"
  principal_id         = azurerm_user_assigned_identity.example.principal_id
}
```

- Powershell script in Azure CLI for **ACRPull** Role

```powershell
# Example data
$resourceGroupName = "xpg_shared_infra"
$containerRegistryName = "eheu2xpgsharedacr"
$roleDefinitionName = "AcrPull"
$objectid = "20ccbd44-a707-4a4f-b105-858b37da391a"  #for this identityName = "eh-aks-1-load-agentpool"

```

```
# Get the Container Registry ID
$scope = az acr show --name $containerRegistryName --resource-group $resourceGroupName --query id --output tsv
echo "scope is $scope"

# Create the Role Assignment
az role assignment create --role $roleDefinitionName --scope $scope --assignee-object-id $objectid --assignee-principal-type ServicePrincipal

```

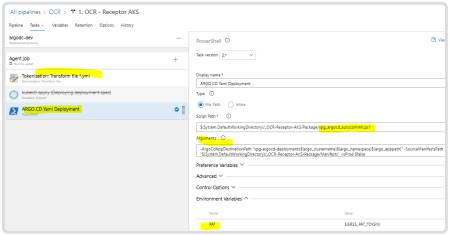
PowerShell Script for Committing to Argo CD Repository

xpg_argocd_au... mit.ps1
05 Dec 2025, 11:03 AM

==========

Documentation for the automated PowerShell Script to deploy the tokenized manifest to the Argo CD Repository in #7

The addition of a PowerShell task following the tokenization step enables automatic commits.



<u>Parameters to be passed to above PowerShell script in release pipeline</u>

ArgoCdXpgRepoUrl (Optional)

- **Value**: `https://$env:PAT@dev.azure.com/eviCoreDev/XPlatform/_git/xpg-argocd-deployments`
- **Details**: Path of ARGOCD URL. Set PAT Token in CICD Task.
- **ArgoCdAppDestinationPath**
  - **Value**: `xpg-argocd-deployments/poc/eh-aks-1-poc/ns-xpg/ocr-services/ocr-receptor-api`
  - **Details**: Destination Path in ARGO CD Repo for YAML Generation.
- **SourceManifestsPath**
  - **Value**: `$(System.DefaultWorkingDirectory)/$(Release.PrimaryArtifactSourceAlias)/Package/Manifests`
  - **Details**: Path of the manifest of the required project.
- **IsProd**
  - **Value**: `$false`
  - **Details**: If `IsProd` is true, the commit will be on the main branch; else it will be on the staging branch of the ARGO CD repo. For custom branches, use the `-CheckoutBranch` option.
- **IsChartPresent (Optional)**
  - **Value**: `"N"`
  - **Details**: If your project contains a chart, set to `Y`; else the default is `N`.
- **HelmValuesFiles (Optional)**
  - **Value**: `$(System.DefaultWorkingDirectory)/$(Release.PrimaryArtifactSourceAlias)/Package/config/Config.yaml`
  - **Details**: You can supply multiple value files in CSV format or a space-separated pattern.

- **Example**: `"D:/config/Config.nTier1.yaml", "D:/config/Config.nTier2.yaml", "D:/config/Config.nTier3.yaml"`

- **UserEmail (Optional)**
  - **Default**: `AbilitantiTechSupport@evicore.com`
  - **Details**: Name of the user to be reflected in Git commit username

How do I upload the Kafka cert to the vault so that it won't change its format?

```
// Login to AZ CLI and you shlud have access on vault
using System;
using System.Threading.Tasks;
using Azure.Identity;
using Azure.Security.KeyVault.Secrets;

const string secretName = "kafka-certs-dev";
var keyVaultName = "eheu2dv1akvocr";
var kvUri = $"https://{keyVaultName}.vault.azure.net";
var client = new SecretClient(new Uri(kvUri), new DefaultAzureCredential(new DefaultAzureCredentialOptions {  }));
var secretValue = File.ReadAllText("D:\\kafkacert.crt");
await client.SetSecretAsync(secretName, secretValue);

```

How to set up SWAGGER UI after ARGO CD Cluster migration.

For Nswag Nuget

```
    public void ConfigureServices(IServiceCollection services)
    {
            services.Configure<ForwardedHeadersOptions>(options =>
           {
             options.ForwardedHeaders =
                    ForwardedHeaders.XForwardedFor | ForwardedHeaders.XForwardedProto;
          });
             .......
    }

    public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
        {
            app.UsePathBase("/ocr-receptorapi");
            app.UseForwardedHeaders();
            app.UseOpenApi(settings =>
            {
                settings.PostProcess = (document, request) =>
                {
                    document.Servers.Clear();
                    document.Servers.Add(new NSwag.OpenApiServer { Url = "/ocr-receptorapi" });
                };
            });
            app.UseSwaggerUi(settings => { settings.DocumentPath = "/swagger/v1/swagger.json"; });
        }
```

For Swashbuckle Nuget

```

            app.UseSwagger(c =>
            {
                c.PreSerializeFilters.Add((swaggerDoc, request) =>
                {
                    if (request.Headers.TryGetValue("X-Forwarded-Prefix", out var serverPath))
                    {
                        swaggerDoc.Servers = new List<OpenApiServer>()
                        {
                            new OpenApiServer() { Url = serverPath }
                        };
                    }
                });
            });
```